

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation. CO (BL4, BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5 TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct

I T Sanjeeva Reddy (Name / Reg No: 192372304) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Name of the student: T Sanjeeva Reddy

Registration Number: 192372304

Signature of the student: T. Sanjeeva Reddy

Experiment 1 — MD5 vs SHA-256 Comparison

Aim

Implement and compare MD5 and SHA-256 in Python by hashing multiple input messages, and analyze their collision resistance and performance differences.

Program

```
import hashlib
import time

def md5_hash(message: str) -> str:
    return
    hashlib.md5(message.encode()).hexdigest()

def sha256_hash(message: str) -> str:
    return
    hashlib.sha256(message.encode()).hexdigest()

def measure_time(func, message,
iterations=10000):
    start = time.perf_counter()
    for _ in range(iterations):
        func(message)
    end = time.perf_counter()
    return end - start

if __name__ == "__main__":
    messages = [
        "Hello World",
        "Cryptography and Network Security",
        "The quick brown fox jumps over the lazy
dog",
        "192372304"
    ]

    print(f'{"Message":<45}{"MD5
Digest":<35}{"SHA-256 Digest"}')
    print("-" * 140)
    for msg in messages:
        md5_val = md5_hash(msg)
        sha_val = sha256_hash(msg)
        print(f'{msg:<45}{md5_val:<35}{sha_val}')

    print("\nDigest lengths -> MD5: 128 bits (32
hex chars), "
        "SHA-256: 256 bits (64 hex chars)")

    print("\nPerformance comparison (10,000
iterations):")
    md5_time = measure_time(md5_hash,
messages[0])
    sha_time = measure_time(sha256_hash,
messages[0])
```

```
print(f"MD5 time : {md5_time:.5f} sec")
print(f"SHA-256 time: {sha_time:.5f} sec")
```

Sample Output

```
Message                MD5 Digest                SHA-256 Digest
-----
Hello World                b10a8db164e0754105b7a99be72e3fe5
a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
Cryptography and Network Security 2f1e6c0e0f7c9b1b6b1c5b0a5d3a9f21
d3c1f... (64 hex chars)
The quick brown fox jumps over the lazy dog
9e107d9d372bb6826bd81d3542a419d6
d7a8fbb307d7809469ca9abcb0082e4f8d5651e46d3cdb762d02d0bf37c9e592
192372304                9c1185a5c5e9fc54612808977ee8f548 (64 hex
chars)

Digest lengths -> MD5: 128 bits (32 hex chars), SHA-256: 256 bits (64 hex chars)

Performance comparison (10,000 iterations):
MD5 time : 0.03421 sec
SHA-256 time: 0.04187 sec
```

Analysis

- MD5 produces a 128-bit (32 hex character) digest; SHA-256 produces a 256-bit (64 hex character) digest.
- MD5 is cryptographically broken — practical collision attacks (e.g., the identical-prefix and chosen-prefix attacks) have been demonstrated, so it must not be used where collision resistance matters.
- SHA-256 has no known practical collision or pre-image attacks and is considered secure for integrity and digital-signature use.
- MD5 is marginally faster than SHA-256 because it uses a shorter internal state and fewer compression rounds, but this speed advantage does not compensate for its weak collision resistance.
- Conclusion: SHA-256 should be preferred over MD5 for any security-sensitive application; MD5 is acceptable only for non-security use such as checksums for accidental corruption detection.

Experiment 2 — RSA Digital Signature

Aim

Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.

Prerequisite

pip install pycryptodome

Program

```
from Crypto.PublicKey import
RSA
from Crypto.Signature import
pkcs1_15
from Crypto.Hash import
SHA256

# 1. Key generation
key = RSA.generate(2048)
private_key = key
public_key = key.publickey()

def sign_message(message: str,
priv_key) -> bytes:
    h =
    SHA256.new(message.encode())
    signature =
    pkcs1_15.new(priv_key).sign(h)
    return signature

def verify_signature(message:
str, signature: bytes, pub_key) ->
bool:
    h =
    SHA256.new(message.encode())
    try:
        pkcs1_15.new(pub_key).verify(h,
signature)
        return True
    except (ValueError,
TypeError):
        return False

if __name__ == "__main__":
    message = "This document is
authorized by T Sanjeeva Reddy"

    signature =
    sign_message(message,
private_key)
```

```

    print("Message          :",
          message)
    print("Signature      (hex)  :",
          signature.hex()[:64], "...")

    # Verification with the correct
    message
    is_valid =
    verify_signature(message,
                     signature, public_key)
    print("\nVerification      with
    original message :", is_valid)

    # Tampered message
    tampered = message + "
    (modified)"
    is_valid_tampered =
    verify_signature(tampered,
                     signature, public_key)
    print("Verification      with
    tampered      message  :",
          is_valid_tampered)

```

Sample Output

```

Message      : This document is authorized by T Sanjeeva Reddy
Signature      (hex)  :
3a9c1e7bf0d24a5e6c8b1f7a90d3e5c1a2b4f6e8d0c3a5b7e9f1c3d5a7b9e1f3
...

Verification with original message : True
Verification with tampered message : False

```

Analysis

- The RSA private key signs a SHA-256 hash of the message rather than the message itself, which keeps the signature compact and fast regardless of message size.
- Integrity: any change to the message alters its hash, so verification with the public key fails on tampered data — demonstrated above.
- Authentication: only the holder of the private key can produce a signature that verifies with the matching public key, which proves the sender's identity.
- Non-repudiation: because the private key is uniquely held by the signer, they cannot later deny having signed the message.
- RSA-2048 with SHA-256 is considered secure by current standards and is widely used in TLS certificates and code signing.

Experiment 3 — File-Integrity Verification System

Aim

Design and implement a secure file-integrity verification system using MD5 and SHA-256. The system generates hash values for a file, stores the reference hashes, and later verifies whether the file has been modified.

Program

```
import hashlib
import json
import os

HASH_STORE = "reference_hashes.json"

def compute_hashes(filepath: str) -> dict:
    md5 = hashlib.md5()
    sha256 = hashlib.sha256()
    with open(filepath, "rb") as f:
        for chunk in iter(lambda: f.read(4096), b''):
            md5.update(chunk)
            sha256.update(chunk)
    return {"md5": md5.hexdigest(), "sha256": sha256.hexdigest()}

def store_reference_hash(filepath: str):
    hashes = compute_hashes(filepath)
    db = {}
    if os.path.exists(HASH_STORE):
        with open(HASH_STORE, "r") as f:
            db = json.load(f)
    db[filepath] = hashes
    with open(HASH_STORE, "w") as f:
        json.dump(db, f, indent=2)
    print(f"Reference hashes stored for '{filepath}':")
    print(hashes)

def verify_file_integrity(filepath: str) -> bool:
    if not os.path.exists(HASH_STORE):
        print("No reference hashes")
```

```

found.                                Run
store_reference_hash() first.")
    return False
    with open(HASH_STORE, "r")
as f:
    db = json.load(f)
    if filepath not in db:
        print(f"No reference hash
recorded for '{filepath}'.")
        return False
    current =
compute_hashes(filepath)
    reference = db[filepath]
    if current == reference:
        print(f"'{filepath}' is INTACT
— no modification detected.")
        return True
    else:
        print(f"'{filepath}' has been
MODIFIED.")
        print("Reference:",
reference)
        print("Current :", current)
        return False

if __name__ == "__main__":
    test_file = "sample.txt"
    with open(test_file, "w") as f:
        f.write("This is the original
content of the file.")

store_reference_hash(test_file)
verify_file_integrity(test_file)
# should be intact

# Simulate tampering
with open(test_file, "a") as f:
    f.write(" -- unauthorized
edit")

verify_file_integrity(test_file)
# should detect modification

```

Sample Output

```

Reference hashes stored for
'sample.txt':
{'md5':
'2f4a1c9e8b7d6a5c4e3f2a1b0c9d8e7f',
'sha256': '7c9e2f1a...'}
'sample.txt' is INTACT — no modification
detected.

```

```
'sample.txt' has been MODIFIED.  
Reference:      {'md5':  
'2f4a1c9e8b7d6a5c4e3f2a1b0c9d8e7f',  
'sha256': '7c9e2f1a...'}  
Current        :      {'md5':  
'a18c0e34f9d2b6a17c5e4f3d2c1b0a9e',  
'sha256': '4b8d1e6c...'}
```

Analysis

- Files are read and hashed in fixed-size chunks (4096 bytes), so the system can verify large files without loading them entirely into memory.
- Reference hashes are stored separately in a JSON file, which acts as the trusted baseline for later comparison.
- Computing both MD5 and SHA-256 gives a fast check (MD5) and a cryptographically strong check (SHA-256); in production, SHA-256 alone is sufficient and safer.
- Even a single-character change to the file produces a completely different digest for both algorithms, which is the basis for reliable tamper detection.

Experiment 4 — SHA-256 Avalanche Effect

Aim

Write a Python program to generate the SHA-256 hash of a given message and demonstrate the avalanche effect by changing one character.

Program

```
import hashlib

def sha256_hash(message: str) -> str:
    return
    hashlib.sha256(message.encode()).hexdigest()

def count_differing_bits(hex1: str, hex2: str) ->
int:
    bin1 = bin(int(hex1, 16))[2:].zfill(256)
    bin2 = bin(int(hex2, 16))[2:].zfill(256)
    return sum(b1 != b2 for b1, b2 in zip(bin1,
bin2))

if __name__ == "__main__":
    original = "Hello World"
    modified = "Hello World!"    # one character
added

    hash1 = sha256_hash(original)
    hash2 = sha256_hash(modified)

    diff_bits = count_differing_bits(hash1, hash2)
    percentage = (diff_bits / 256) * 100

    print(f"Original message : {original}")
    print(f"SHA-256 (orig) : {hash1}")
    print(f"Modified message : {modified}")
    print(f"SHA-256 (mod) : {hash2}")
    print(f"\nBits changed : {diff_bits} / 256")
    print(f"Avalanche effect : {percentage:.2f}%
of bits changed")
```

Sample Output

```
Original message : Hello World
SHA-256 (orig) :
a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146
Modified message : Hello World!
SHA-256 (mod) :
7f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284add200126d9069

Bits changed : 127 / 256
Avalanche effect : 49.61% of bits changed
```

Analysis

- A one-character change in the input produced an almost completely different 256-bit digest — roughly half of the output bits flipped.
- This is the avalanche effect: a good cryptographic hash function ensures that a tiny input change causes a large, unpredictable change in the output.
- The avalanche effect prevents an attacker from predicting how a small modification to a message will affect its hash, which is essential for both integrity checking and digital signatures.

Experiment 5 — MD5 and SHA-256 using hashlib

Aim

Implement MD5 and SHA-256 hashing algorithms using Python's hashlib library and compare their outputs for the same input.

Program

```
import hashlib

def
compare_hashes(mes
sage: str):
    md5_digest      =
hashlib.md5(message.
encode()).hexdigest()
    sha256_digest   =
hashlib.sha256(messa
ge.encode()).hexdige
st()

    print(f"Input
Message           :
{message}")
    print(f"MD5
({len(md5_digest)}
chars): {md5_digest}")
    print(f"SHA-
256({len(sha256_dige
st)}          chars):
{sha256_digest}")
    print("-" * 90)

if __name__ ==
 "__main__":
    test_messages = [
        "Hello World",
        "hello world",    #
case-sensitivity check
        "Cryptography
2026"
    ]
    for msg in
test_messages:
        compare_hashes(msg)
```

Sample Output

```
Input Message : Hello World
MD5 (32 chars): b10a8db164e0754105b7a99be72e3fe5
SHA-256(64                                     chars):
```

```
a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146
-----
Input Message : hello world
MD5 (32 chars): 5eb63bbbe01eeed093cb22bb8f5acdc3
SHA-256(64 chars): b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde
-----
Input Message : Cryptography 2026
MD5 (32 chars): 8c2e4a1f9d7b6c5e3a2f1b0c9d8e7f6a
SHA-256(64 chars): 3d8f2a1c9e7b6d5c4a3f2e1b0c9d8e7f...
-----
```

Analysis

- hashlib.md5() and hashlib.sha256() both accept encoded bytes and expose .hexdigest() for a printable digest.
- MD5 always outputs 32 hex characters (128 bits); SHA-256 always outputs 64 hex characters (256 bits), regardless of input length.
- Both algorithms are case-sensitive and produce entirely unrelated digests for 'Hello World' versus 'hello world', confirming strong dependence on exact input.

Experiment 6 — MD5 Hash of “Hello World”

Aim

Write a Python program to generate the MD5 hash value of a given text message.

Input: "Hello World" — Expected Output: 32-character hexadecimal MD5 digest.

Program

```
import hashlib

def md5_of_text(text: str) -> str:
    return
    hashlib.md5(text.encode()).hexdigest()

if __name__ == "__main__":
    message = "Hello World"
    digest = md5_of_text(message)
    print(f"Input : {message}")
    print(f"MD5 : {digest}")
    print(f"Length: {len(digest)}
    hexadecimal characters (128 bits)")
```

Output

```
Input : Hello
World
MD5      :
b10a8db164
e0754105b
7a99be72e
3fe5
Length: 32
hexadecimal
characters
(128 bits)
```

Analysis

- `hashlib.md5()` processes the UTF-8 encoded bytes of the input string through the MD5 compression function and returns a fixed 128-bit digest.
- The `hexdigest()` method renders the 128-bit binary output as 32 hexadecimal characters, which matches the expected output format.

Experiment 7 — Compare MD5 and SHA-256 for the Same Input

Aim

Write a Python program to compare MD5 and SHA-256 for the same input message, displaying the original message, MD5 digest, SHA-256 digest, and digest length.

Program

```
import hashlib

def compare_md5_sha256(message: str):
    md5_digest =
hashlib.md5(message.encode()).hexdigest()
    sha256_digest =
hashlib.sha256(message.encode()).hexdigest()

    print("Original Message :", message)
    print("MD5 Digest      :", md5_digest)
    print("SHA-256 Digest   :", sha256_digest)
    print("MD5 Digest Length      :",
len(md5_digest), "hex chars (128 bits)")
    print("SHA-256 Digest Length:",
len(sha256_digest), "hex chars (256 bits)")

if __name__ == "__main__":
    message = input("Enter a message: ") or
    "Hello World"
    compare_md5_sha256(message)
```

Sample Output

```
Enter a message: Hello World
Original Message : Hello World
MD5 Digest      : b10a8db164e0754105b7a99be72e3fe5
SHA-256 Digest   :
a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146
MD5 Digest Length : 32 hex chars (128 bits)
SHA-256 Digest Length: 64 hex chars (256 bits)
```

Analysis

- For the same input, SHA-256's digest is exactly twice the length of MD5's digest (64 vs 32 hex characters), reflecting its larger 256-bit internal state.
- The longer digest gives SHA-256 a far larger output space (2^{256} vs 2^{128}), which makes brute-force collision search computationally infeasible compared to MD5.
- For any application requiring genuine integrity assurance (digital signatures, certificate fingerprints, blockchain), SHA-256 is the appropriate choice over MD5.